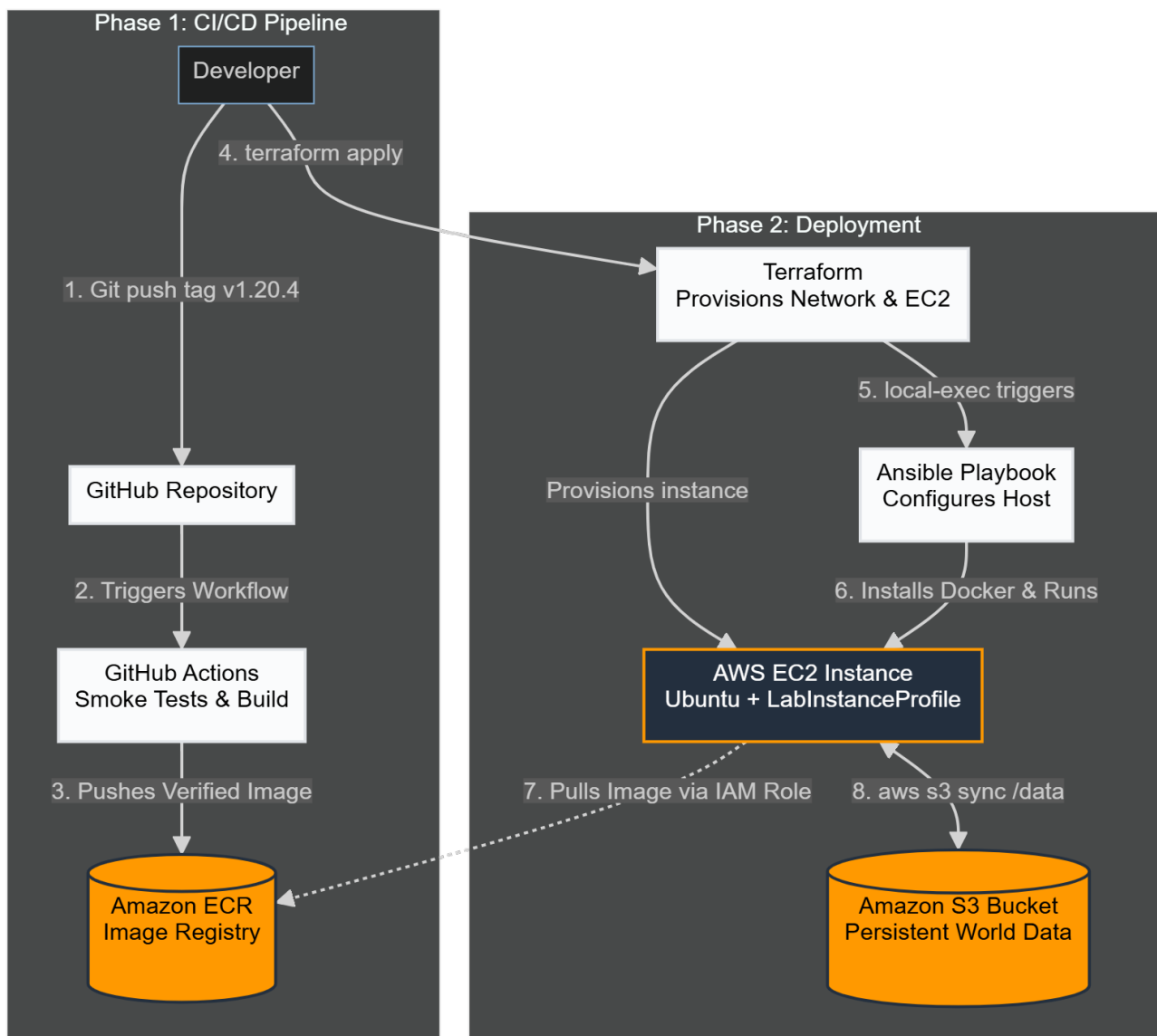


Infrastructure Automation Runbook

Dean Leon | 934513949
Infrastructure Automation
CS 312 System Administration

1. Runtime Architecture

The automated deployment pipeline consists of two primary flows: an image publishing pipeline via GitHub Actions and infrastructure provisioning via Terraform and Ansible. The EC2 instance is deployed with the `LabInstanceProfile`, allowing it to seamlessly authenticate to AWS ECR to pull the pinned Docker image and AWS S3 to restore persistent world data before the container starts.



Architecture diagram generated using Mermaid.js

2. Terraform Variables and Configuration

Infrastructure is modeled strictly as code using Terraform to guarantee reproducible and auditable environments. The following variables define the provisioning parameters:

- `aws_region` : Specifies the target AWS region for deployment (e.g., `us-east-1`).
- `instance_type` : Controls the compute capacity of the EC2 instance (set to `t3.medium` to support the memory requirements of the Minecraft container).
- `key_name` : The identifier of the AWS key pair (e.g., `cs312-key`) used to associate the public SSH key with the newly provisioned host.
- `private_key_path` : The local file path pointing to the `.pem` file used by the `local-exec` provisioner to authenticate Ansible's initial configuration run against the new EC2 instance.

State Management: Terraform state is stored locally in `terraform.tfstate` . This file is excluded from version control via `.gitignore` to prevent accidental exposure of resource IDs and sensitive outputs. The operator running `terraform apply` or `terraform destroy` is responsible for retaining the state file between operations. A remote S3 backend with state locking is not configured in this environment due to AWS Academy credential constraints, but would be the recommended approach for a production deployment.

3. Infrastructure Change Process

To ensure infrastructure auditability and prevent undocumented configuration drift, all modifications must flow through the version control system:

1. **Branching:** A teammate creates a feature branch from `main` (e.g., `feature/increase-ebs-storage`).
2. **Code Modification:** The teammate updates the state declarations in the Terraform files or task definitions in the Ansible playbook.
3. **Review & PR:** A Pull Request is opened against `main` . The team reviews the proposed changes and validates the dry-run output of `terraform plan` .
4. **Merge & Apply:** Once approved, the PR is merged. An authorized operator pulls the updated `main` branch locally and applies the new state using `terraform apply` .

4. Cost-Prevention Teardown Checklist

To prevent runaway resource costs in the AWS Academy Learner Lab environment, the infrastructure stack must be systematically destroyed when operations conclude.

Mandatory Teardown Sequence:

- **1. Destroy Compute Infrastructure:** Run `terraform destroy -auto-approve` to permanently terminate the EC2 instance, its root EBS volume, and associated Security Groups.
- **2. Purge Container Images:** Navigate to the AWS Elastic Container Registry console, empty the `minecraft-server` repository of all published images, and delete the repository.
- **3. Purge Backup Data:** Navigate to the AWS S3 console, open the persistent world backup bucket, permanently delete all contained backup archives, and delete the bucket.
- **4. End Session:** Terminate the AWS Learner Lab session explicitly using the portal's "End Lab" function to revoke temporary credentials.

5. World-Data Recovery Strategy

World data persistence is achieved by backing up the Minecraft `/data` directory to a dedicated S3 bucket. The Ansible playbook is responsible for restoring from this backup before container start, ensuring players return to the same world after any rebuild.

- **Backup location:** World data is stored in an S3 bucket (e.g., `s3://minecraft-world-backup-<account-id>/world/`). The EC2 instance writes to this bucket via the permissions granted by `LabInstanceProfile` — no credentials are stored on disk.
- **Backup trigger:** The operator runs `aws s3 sync /data s3://<bucket>/world/` before tearing down the instance, or a cron job on the host can perform periodic backups automatically.
- **Restore on rebuild:** The Ansible playbook executes `aws s3 sync s3://<bucket>/world/ /data` before starting the Minecraft container. The restore runs first so the server loads the existing world rather than generating a new one. If the S3 path is empty (first deploy), the server generates a fresh world as expected.
- **Idempotency:** `aws s3 sync` only transfers changed or missing files, so re-running the playbook against a live host does not overwrite an in-progress world with a stale backup.
- **Rebuild outcome:** As demonstrated in Checkpoint 4 of the video, a full `terraform destroy` followed by `terraform apply` and the Ansible playbook restores the same world — no player progress is lost.

6. Video Demonstration & Rebuild Proof

A video demonstration verifying the infrastructure automation, pipeline execution, and world-data recovery strategy can be found here:

https://media.oregonstate.edu/media/t/1_xzwcuvd2

Note: The video was edited and sections of down time, i.e., waiting for the ec2 instance to reboot, were cut out.

- **Checkpoint 1 (00:14):** `terraform apply` execution and automated Ansible playbook configuration.
- **Checkpoint 2 (00:57):** Service validation via `nmap`, proving the port is open and the custom MOTD is active.

- **Checkpoint 3 (01:37):** Verification of the GitHub Actions CI/CD pipeline triggering on a tag push and passing smoke tests.
- **Checkpoint 4 (02:01):** Simulated server loss via `terraform destroy`, followed by an automated rebuild proving the S3 restoration of existing world data.

Document Attribution:

I used LLM assistance to format this document and to validate the technical accuracy of the content.

<https://github.com/CS-312-001-S2026/minecraft-ops-GenericProgram>